

1 Introduction

Cette deuxième section est centrée sur l'algorithme de César, un algorithme simple de cryptographie symétrique.

2 Algorithme de César

2.1 Principe de l'algorithme

Le principe de cet algorithme est le suivant.

Données d'entrée :

- Le message à chiffrer est une chaîne de caractères mais l'algorithme ne chiffrera que les lettres de ce message. Tous les autres caractères (chiffres, caractères spéciaux...) seront gardés tels quel dans le message chiffré.
- La clé est un entier positif ou négatif.

Principe du chiffrement :

Le message est chiffré caractère par caractère. Chaque caractère est "décalé" dans l'alphabet d'un nombre de places égal à la clé. On considère que l'alphabet est "cyclique", c'est-à-dire que 'A' suit 'Z' et que 'Z' précède 'A'. Ainsi si 'Y' est décalé de 3, il devient un 'B'. Le chiffrement conserve la casse (majuscule ou minuscule) des lettres et ne modifie pas les autres caractères.

Exemples :

Le message *Ab!xY*, chiffré avec la clé 5 devient *Fg!cD*.

Le message *Ab!xY*, chiffré avec la clé -33 devient *Tu!qR*.

2.2 Structure

Il existe de nombreuses façon de formaliser l'algorithme de César. Nous allons adopter une vision généralisable à des versions plus complexes de l'algorithme (voir partie 2). Cette vision nous amènera à implémenter une étape intermédiaire dont on pourrait se passer dans la version simple de l'algorithme.

1. Dans le répertoire *Algorithmes/Realisations*, créez une classe *AlgorithmeCesar* implémentant l'interface *IAlgorithme*.
2. Ajoutez à cette classe les deux méthodes suivantes (que l'on implémentera plus tard).

```
public int CharToInt(char c)
```

```
public char ChiffrerChar(char c, int cle)
```

La méthode **CharToInt** renvoie la position de *c* dans l'alphabet. Par exemple, pour 'a' ou 'A', elle renverra 0, pour 'b' ou 'B', elle renverra 1, pour 'y' ou 'Y', elle renverra 24.

La méthode **ChiffrerChar** est chargée de chiffrer un caractère à l'aide de la clé donnée en paramètre. Pour cela, si le caractère est une lettre :

- on utilisera la méthode **CharToInt** pour transformer ce caractère en un nombre entre 0 et 25,
- on effectuera une opération mathématique simple pour calculer la position du caractère chiffré dans l'alphabet à partir de cette valeur
- on en déduira le caractère chiffré en prenant en compte la casse du caractère d'entrée.

On suppose dans cette méthode que la clé est comprise entre 0 et 25. On s'occupera de normaliser la clé ailleurs.

Indication : les méthodes `char.isLower` et `char.isUpper` permettent de savoir si un caractère est une majuscule ou une minuscule. La méthode `char.isLetter` permet de savoir si un caractère est une lettre.

Pour le chiffrement, il faudra penser à normaliser la clé, c'est-à-dire à la ramener entre 0 et 25 à l'aide d'une opération mathématique simple. En effet, une clé de 27 correspond à une clé de 1 et une clé de -5 correspond à une clé de 21. Il ne restera alors qu'à chiffrer les caractères un par un.

2.3 Challenges

Challenge CES1 : Conversion char/int :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'une lettre (majuscule ou minuscule)
- attend la position de cette lettre dans l'alphabet
- validera ou non votre réponse par l'envoi de OK ou NOK|...

3. Codez la méthode *CharToInt* de la classe *AlgorithmeCesar*.

4. Dans le répertoire *Challenges/Realisations*, créez une classe *ChallengeCES1* implémentant ce challenge.

5. Ajoutez ce nouveau challenge à votre fabrique.

6. Vérifiez que vous validez le premier challenge de la section.

Challenge CES2 : Chiffrement d'un caractère :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'un caractère (majuscule ou minuscule ou spécial)
- envoie de la clé (entier entre 0 et 25)
- attend le caractère chiffré associé
- validera ou non votre réponse par l'envoi de OK ou NOK|...

7. Codez la méthode *ChiffrerChar* de la classe *AlgorithmeCesar*.

8. Dans le répertoire *Challenges/Realisations*, créez une classe *ChallengeCES2* implémentant ce challenge.

9. Ajoutez ce nouveau challenge à votre fabrique.

10. Vérifiez que vous validez le deuxième challenge de la section.

Challenge CES3 : Chiffrement classique :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'une chaîne de caractères (majuscule ou minuscule ou spécial) de longueur aléatoire
- envoie de la clé (entier positif ou négatif)
- attend la chaîne de caractères chiffrée
- validera ou non votre réponse par l'envoi de OK ou NOK|...

11. Codez la méthode *Chiffrer* de la classe *AlgorithmeCesar*.
12. Dans le répertoire *Challenges/Realisations*, créez une classe *ChallengeCES3* implémentant ce challenge.
13. Ajoutez ce nouveau challenge à votre fabrique.
14. Vérifiez que vous validez le troisième challenge de la section.

Challenge CES4 : Déchiffrement classique :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'une chaîne de caractères (majuscule ou minuscule ou spécial) de longueur aléatoire
- envoie de la clé (entier positif ou négatif) ayant servi au chiffrement de cette chaîne de caractères
- attend la chaîne de caractères déchiffrée
- validera ou non votre réponse par l'envoi de OK ou NOK|...

15. Codez la méthode *Chiffrer* de la classe *AlgorithmeCesar*.
16. Dans le répertoire *Challenges/Realisations*, créez une classe *ChallengeCES4* implémentant ce challenge.
17. Ajoutez ce nouveau challenge à votre fabrique.
18. Vérifiez que vous validez le quatrième challenge de la section.

3 Bonus - Algorithme de César double

Cette partie n'est pas attendue dans vos rendus mais est un bon exercice montrant en particulier une astuce classique de cryptographie pour convertir une chaîne de caractère en nombre qui est utilisée dans de nombreux algorithmes modernes.

3.1 Principe de l'algorithme

L'algorithme de César double est une variante classique de l'algorithme de César. Il consiste à considérer les caractères 2 par 2 au lieu de 1 par 1. Le décalage est alors réalisé sur chaque paire de caractères. Ainsi avec une clé de 3, la paire AA devient AD, la paire AZ devient BC et la paire ZY devient AB. L'algorithme ne fonctionne que sur des messages de taille paire ne contenant que des lettres (il respecte bien évidemment la casse).

3.2 Structure

Nous allons adopter une structure similaire à celle de notre algorithme de César.

19. Dans le répertoire *Algorithmes/Realisations*, créez une classe *AlgorithmeCesarDouble* implémentant l'interface *IAlgorithme*.
20. Ajoutez à cette classe les deux méthodes suivantes (que l'on implémentera plus tard).

```
public int CharsToInt(char c1, char c2)
```

```
public string ChiffrementPairChars(char c1, char c2, int cle)
```

La méthode **CharsToInt** renvoie la place de la paire (c1,c2) dans la liste des paires possible. Par exemple, pour AA (ou aA...) , elle renverra 0, pour AB (ou 'aB'...), elle renverra 1, pour 'ZY', elle renverra 674. Cette méthode peut sembler un peu délicate mais est en fait très simple si l'on adopte la bonne vision. L'idée est de voir cette conversion comme une conversion similaire à la conversion d'un nombre hexadécimal vers un nombre décimal.

En effet, en hexadécimal, on utilise un "alphabet" de 16 chiffres (0,1,2,...,9,a,b,c,d,e,f) ayant pour "valeurs" respectives 0,1,2,3...,9,10,11,12,13,14,15. Le nombre hexadécimal *ac* se convertit en faisant le calcul suivant : $10 \times 16 + 12$ (valeur de *a* * base + valeur de *c*).

La méthode **ChiffrementPairChars** est chargée de chiffrer la paire de caractères à l'aide de la clé donnée en paramètre. Pour cela,

- on utilisera la méthode **CharsToInt** pour transformer la paire de caractères en un nombre entre 0 et 675,
- on effectuera une opération mathématique simple pour calculer le nombre associé à la paire chiffrée à partir de cette valeur
- on en déduira la paire de caractères chiffrés en prenant en compte la casse de chacun des caractères d'entrée.

On suppose dans cette méthode que la clé est comprise entre 0 et 675. On s'occupera de normaliser la clé dans la méthode de chiffrement.

3.3 Challenge

Challenge CES5 : Conversion chars/int :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'une chaîne de 2 caractères exactement (majuscules ou minuscules)
- attend la position de cette paire de caractères dans la liste des paires
- validera ou non votre réponse par l'envoi de OK ou NOK|...

21. Codez la méthode *CharsToInt* de la classe *AlgorithmeCesarDouble*.

22. Dans le répertoire *Challenges/Realisations*, créez une classe *ChallengeCES5* implémentant ce challenge.

23. Ajoutez ce nouveau challenge à votre fabrique.

24. Vérifiez que vous validez le cinquième challenge de la section.

Challenge CES6 : Chiffrement double :

Le serveur enverra un nombre aléatoire de tests et terminera la discussion par "FIN". Pour chaque test le serveur :

- envoie d'une chaîne de caractères de taille paire (majuscules ou minuscules)
- envoie de la clé (entier positif ou négatif)
- attend la chaîne de caractères chiffrée
- validera ou non votre réponse par l'envoi de OK ou NOK|...

25. Codez la méthode *ChiffrementPairChars* de la classe *AlgorithmeCesarDouble*.

26. Codez la méthode *Chiffrer* de la classe *AlgorithmeCesarDouble*.

27. Dans le répertoire *Challenges/Realisations*, créez une classe *ChallengeCES6* implémentant ce challenge.

28. Ajoutez ce nouveau challenge à votre fabrique.

29. Vérifiez que vous validez le dernier challenge de la section.

30. Codez la méthode *Déchiffrer* de la classe *AlgorithmeCesarDouble*.